Chapter 6

First-Stage Ranking: From Candidates to a Shortlist

6.1 From Retrieval to Ranking

In the multi-stage recommender architecture introduced in Chapter 2, retrieval casts a wide net to surface thousands of potentially relevant candidates from a corpus of millions. But retrieval models optimize for recall at the expense of precision. They use lightweight representations (embeddings, approximate nearest neighbors) that cannot capture the rich, nuanced signals needed to produce a truly personalized ordering.

The ranking stage is where precision takes center stage. Operating on a much smaller candidate set (typically hundreds of items), the ranker can afford to use richer feature sets, more expressive models, and cross-features between the user and item that would be computationally prohibitive at retrieval scale.

Instagram Explore, for example, uses a four-stage[1] architecture: retrieval stage selections few 1000 candidates, a first-stage ranker (lightweight model) filters them down to few 100s, a second-stage ranker (heavy neural model) scores those precisely, and a value function combines multi-task predictions into a final score. This chapter focuses on the first-stage ranker: a model that is more expressive than retrieval but still efficient enough to score hundreds of candidates within tight latency budgets.

6.2 Learning to Rank Fundamentals

Before diving into specific models, it is important to understand how we frame the ranking problem for training. The Learning to Rank (LTR) framework provides 3 distinct approaches to this problem.

6.2.1 Pointwise Approach

The pointwise approach treats ranking as independent prediction. Each user-item pair receives a score (typically a click probability or relevance estimate), and items are sorted by these scores. This is conceptually the simplest approach: train a binary classifier on implicit feedback (1 for click, 0 for no-click), and rank by predicted score.

[1] If you are starting with a completely new recsys stack, try with a single ranking stage before moving to separate light vs heavy rerankers.

- Straightforward to implement using standard classification losses (cross-entropy, log loss)
- Allows use of standard evaluation metrics (AUC, log loss) during training
- Each item receives an independent score with no awareness of what other candidates are in the list. Does not directly optimize the ordering of items relative to each other.

Even if the pointwise score is not calibrated to be the probability of a click, pointwise approaches work well in practice because ranking only requires monotonic scores: if P(click|item A) > P(click|item B), the ranker will place A above B regardless of the absolute probability values

This is a common starting point in industry: many production systems use pointwise ranking with logistic regression or gradient boosted trees. The below code snippet shows a simple classifier using XGB.

```
# Pointwise approach: standard binary classification
import xgboost as xgb

model = xgb.XGBClassifier(
    objective='binary:logistic',   # Pointwise: independent P(click) per item
    eval_metric='auc',
    n_estimators=100, max_depth=6, learning_rate=0.1
)
model.fit(X_train, y_train, eval_set=[(X_val, y_val)])

# At inference: sort candidates by predicted probability
proba = model.predict_proba(X_test)[:, 1]
ranked_items = item_ids[np.argsort(-proba)]
```

6.2.2 Pairwise Approach

Pairwise methods train the model to prefer relevant items over irrelevant ones within the same context. Given a pair (positive item, negative item) for a user, the model learns to assign a higher score to the positive. This approach directly optimizes the relative ordering.

- **RankNet:** Uses a logistic loss on score differences to learn pairwise preferences
- **LambdaRank:** Weights pairwise gradients by the change in NDCG that swapping two items would produce, effectively optimizing NDCG through pairwise comparisons
- **LambdaMART:** Combines LambdaRank with gradient boosted trees (MART), widely considered the gold standard for feature-based ranking

LambdaMART deserves a deeper look because it underpins XGBoost's rank:pairwise objective that we use in the companion exercises.Rather than defining a loss function and deriving gradients from it, LambdaMART defines the gradients directly.

Traditional approaches to optimizing NDCG face a fundamental obstacle: NDCG is a step function of the ranking permutation, so it has zero gradients almost everywhere. LambdaRank sidesteps this by defining pseudo-gradients (called lambdas) for each document pair:

*lambda_ij = sigma_ij * |delta_NDCG_ij|*

where sigma_ij is the logistic gradient for the pair (i, j) based on their score difference, and |delta_NDCG_ij| is the absolute change in NDCG that would result from swapping items i and j in the current ranking. The logistic component provides the direction (which item should rank higher), while the NDCG delta provides the magnitude (how much the ranking metric cares about this particular swap).

This weighting is what makes LambdaMART effectively listwise despite using pairwise comparisons: swapping two items near the top of the list produces a larger delta_NDCG than swapping two items at position 50 and 51, so the model focuses its capacity on getting the top of the ranking right.

LambdaMART combines these lambda gradients with gradient boosted decision trees (Multiple Additive Regression Trees, or MART). At each boosting iteration, a new tree is fit to the lambda gradients, refining the model's ranking. The result is a model that optimizes NDCG indirectly but effectively, without ever defining a differentiable loss function.

In the tutorial accompanying this chapter, LambdaMART is implemented through the XGBRanker(). If you are using the dmlc library, the core logic is as follows:

```
# Pairwise approach: LambdaMART via XGBRanker
model = xgb.XGBRanker(
    objective='rank:pairwise',     # LambdaMART: optimizes NDCG via pairwise lambdas
    n_estimators=100, max_depth=6, learning_rate=0.1
)
# group array tells XGBoost which items belong to the same query
# e.g., [5, 3, 8] means first 5 rows are group 1, next 3 are group 2, etc.
model.fit(X_train, y_train, group=train_groups,
          eval_set=[(X_test, y_test)], eval_group=[test_groups])

# At inference: scores are relative (not probabilities), but ordering is the same
scores = model.predict(X_test)
ranked_items = item_ids[np.argsort(-scores)]
```

An important nuance to keep in mind: the pairwise constraint exists only during training. When forming the training data, all valid pairs within a query group must be constructed, requiring O(n^2) comparisons per group. At inference time, however, the trained model scores each candidate independently, just like a pointwise model. XGBoost's LambdaMART implementation, for example, produces a single scalar score per item at inference. The pairwise comparison was absorbed into the learned gradient structure during training. This makes pairwise models just as fast to serve as pointwise models.

Concretely, XGBoost's rank:pairwise objective does not enumerate every possible pair within a query group, which would be prohibitive for large candidate groups. Instead, for each boosting round it samples a bounded number of candidate pairs per positive item. The modern LambdaRank API (controlled via the lambdarank_num_pair_per_sample and lambdarank_pair_method parameters, added in XGBoost 2.0; see the XGBoost Learning to Rank tutorial in the DMLC documentation) lets you choose between uniform random sampling and topk sampling, where negatives are drawn preferentially from items the current model already scores highly. Topk sampling is the pairwise analogue of hard-negative mining: it forces the model to spend its gradient budget on the pairs it is currently getting wrong, rather than on easy pairs it already ranks correctly. In practice this keeps training time roughly linear in the number of positives per group, even for groups containing hundreds of candidates.

6.2.3 Listwise Approach

Listwise methods consider the entire candidate list and optimize a metric (such as NDCG) directly. These approaches work with differentiable approximations to rank metrics or use techniques like softmax over permutation probabilities.

- LambdaRank is often considered listwise because it uses the full list's NDCG gradient via pairwise swaps weighted by NDCG change
- More complex to implement and require careful handling of the permutation space
- In practice, LambdaMART (which uses LambdaRank-style gradients in a GBDT) is the most common production-grade listwise approach

Approach	Training Signal	Strengths	Limitations
Pointwise	Individual item scores	Simple, standard ML pipeline	Ignores list-level ordering
Pairwise	Item-pair preferences	Directly optimizes ordering	O(n^2) pairs, sampling needed
Listwise	Full candidate list	Optimizes ranking metrics	Complex implementation

6.3 Feature Engineering for Ranking

The quality and coverage of input features remains one of the most impactful levers for ranking model performance. A good feature set captures the user's preferences, the item's characteristics, and the context of the interaction.

6.3.1 Feature Categories

User features capture the user's historical behavior and preferences. These include aggregate statistics (total interactions, average rating given, genre distribution), recency-weighted signals (items interacted with in the last hour vs. last month), and demographic attributes where available.

Item features describe the candidate being scored. These include content metadata (genre, category, price), popularity statistics (global click-through rate, trending score), and quality signals (average rating, return rate).

Context features capture the circumstances of the interaction. Time of day, day of week, device type, and session position all influence what a user is likely to engage with.

Cross-features are derived from the intersection of user and item attributes. For example, 'user's historical click rate on this item category'. These are where the ranking stage gains its advantage over retrieval, which typically cannot compute such features at scale.

As discussed in earlier chapters, the engineered features are stored in a low-latency data store for online retrieval, usually through a Feature Store (which has the additional advantage of version control, etc.). However, for our companion tutorials, we encapsulate this logic in a reusable feature builder class that learns the statistics on the training split and then applies the same aggregates to validation and test splits. The snippet below shows how it fits into the training pipeline:

```
from models import HistoricalFeatureBuilder

# Compute historical aggregate features from training data
builder = HistoricalFeatureBuilder()
builder.fit(train_df)  # Learns user/item/user-item statistics

# Apply to all splits (features are from training data only -- no leakage)
train_df = builder.transform(train_df)
test_df = builder.transform(test_df)

# New features: user_avg_completion, item_total_plays, has_listened_before, etc.
```

6.3.2 Encoding High-Cardinality Features

Features like user IDs and item IDs have cardinalities in the millions. Direct one-hot encoding is infeasible. Common strategies include:

- **Frequency encoding:** Replace a categorical value with its frequency count in the training data. This converts a high-cardinality feature into a single numeric value while preserving popularity information.
- **Target encoding:** Replace a categorical value with the average target value (e.g., average CTR) for that category. Requires careful regularization to avoid overfitting on rare categories.
- **Hashing:** Apply a hash function to map categories to a fixed number of buckets. Introduces some collisions but keeps dimensionality bounded.
- **Embeddings:** Learn a dense vector for each category during model training. This is the standard approach for deep learning models (covered in Chapter 7) but less applicable to tree-based models.

6.3.3 Temporal Feature Engineering

Temporal features deserve special attention in ranking because user intent is highly time-dependent. The same user may want different content at different times.

- Time-based aggregations: user's click rate in the last hour, last day, last week
- Cyclical encoding: hour of day and day of week encoded as sine/cosine pairs to capture periodicity
- Recency signals: time since user's last interaction with this item category
- Session features: position within the current session, number of items already viewed

6.4 Gradient Boosted Trees for Ranking

Gradient Boosted Decision Trees (GBDTs), particularly XGBoost and LightGBM implementations, have been workhorses for ranking in production recommender systems since XGBoost’s release in 2014 (Chen & Guestrin, KDD 2016) and LightGBM in 2017 (Ke et al., NeurIPS 2017). They offer a compelling balance of accuracy, training speed, interpretability, and ease of deployment that makes them an ideal starting point for first-stage ranking.

6.4.1 Why Tree-Based Models for First-Stage Ranking

- Handle heterogeneous feature types naturally (numerical, categorical, derived) without extensive preprocessing

- Capture non-linear feature interactions through tree splits without manual feature crosses
- Provide built-in feature importance, aiding model interpretability and feature debugging
- Fast inference: scoring hundreds of candidates takes single-digit milliseconds
- Support ranking-specific objectives out of the box (LambdaMART for NDCG optimization)
- Require less hyperparameter tuning than deep learning models and often work well with less training data

Many production systems start with (and sometimes stay with) gradient boosted tree rankers. The overhead of deploying and maintaining a deep learning model is only justified when the tree model's expressive limitations become a measurable bottleneck.

6.4.2 Training an XGBoost Ranker

XGBoost implementation in the dmlc library supports multiple ranking objectives. The choice depends on whether you need calibrated probabilities or optimized list ordering:

- **binary:logistic (pointwise):** Predicts P(click | user, item, context). Simple, produces calibrated probabilities, works well as a first baseline.
- **rank:pairwise (LambdaMART):** Optimizes NDCG by comparing pairs of items within the same query group. Requires a group parameter that tells XGBoost which items belong to the same user/session context.
- **rank:ndcg:** Directly optimizes a proxy of NDCG. Similar to rank:pairwise but with a different loss formulation.

6.4.3 The Query Group Abstraction

A critical design choice for ranking models is how you define the 'query group': the set of items that should be compared against each other. For pairwise and listwise objectives, the model only compares items within the same group.

- In non-personalized search use cases where the user types in a keyword(s), the query group is the search query: all documents retrieved for the same query are compared

To make this concrete, consider a search use case where a user types 'wireless headphones.' The query group includes all candidate products retrieved for that query, and the model learns to rank them relative to each other:

Table 6.1. An example of a search ranking training set. Each row is one (query, candidate) pair. Items with the same query_id form one query group; the graded label (0/1/2) is what LambdaMART optimizes via pairwise comparisons within a group.

query_id	query_text	product_id	price	rating	label
q1	wireless headphones	SKU_101	29.99	4.2	0
q1	wireless headphones	SKU_204	79.99	4.7	2
q1	wireless headphones	SKU_305	149.99	4.5	1
q1	wireless headphones	SKU_417	59.99	4.0	0
q2	laptop stand	SKU_088	24.99	3.9	0
q2	laptop stand	SKU_192	39.99	4.4	1
q2	laptop stand	SKU_310	54.99	4.6	2

Similarly, in a recommendation use case, the query group contains all candidates shown to the same user in the same context. The below code snippet shows a dummy example from streaming:

Table 6.2. An example of a recs ranking training set. Each row is one (user, candidate) pair. Items with the same user_id form one group. The label can be ordinal or binary

User_id	item_id	User_genre_pref_score	Item_popularity	Listened (label)
u_42	'track_A'	0.65	0.82	1
u_42	'track_B'	0.20	0.45	0
u_42	'track_C’	0.65	0.33	1
u_42	'track_D’	0.05	0.91	0
u_42	'track_E’	0.20	0.67	1
u_77	'track_F’	0.70	0.55	1
u_77	'track_G’	0.15	0.28	0
u_77	'track_H’	0.70	0.41	0

Group size affects training: too large and you waste computation on easy comparisons; too small and the model lacks sufficient positive/negative contrast

Author's note: In the Yambda tutorial, we use the Global Temporal Split (GTS) protocol from the Yambda paper: 300 days of training data, a 30-minute gap to prevent leakage, and 1 day of test data.

6.5 DeepFM: Automating Feature Crosses

While XGBoost handles feature interactions through tree splits, it relies on the model to discover them during training. DeepFM (Guo et al., 2017) takes a different approach: it explicitly models all pairwise feature interactions through a Factorization Machine (FM) component, combined with a deep neural network for higher-order patterns.

6.5.1 Architecture

DeepFM has two parallel components that share the same input embeddings:

- **FM component:** Assigns each feature an embedding vector and computes all pairwise dot products between these vectors. This explicitly captures second-order feature interactions (e.g., 'user_age x item_genre') without manual feature engineering.
- **Deep component:** Concatenates all feature embeddings and passes them through several fully connected layers. This captures higher-order, implicit interactions.

The FM and deep outputs are summed and passed through a final sigmoid to produce a click probability. The key insight is that both components share embedding layers, so the FM's pairwise interactions and the deep network's higher-order patterns are learned jointly.

```
class FMLayer(nn.Module):
    """Factorization Machine: efficient 2nd-order feature interactions.
    FM output = 0.5 * (||sum(v_i)||^2 - sum(||v_i||^2))
    """
    def forward(self, embeddings):  # (batch, num_fields, embed_dim)
        square_of_sum = torch.sum(embeddings, dim=1) ** 2
        sum_of_square = torch.sum(embeddings ** 2, dim=1)
        return 0.5 * torch.sum(square_of_sum - sum_of_square, dim=1, keepdim=True)

class DeepFM(nn.Module):
    def forward(self, sparse_inputs, dense_inputs=None):
        # 1st order: linear embeddings
        linear_out = sum(self.linear_embeddings[f](idx) for f, idx in
sparse_inputs.items())
        # 2nd order: FM pairwise interactions
        stacked = torch.stack([self.embeddings[f](idx) for f, idx in
sparse_inputs.items()], dim=1)
        fm_out = self.fm(stacked)
        # Higher order: DNN on concatenated embeddings
        dnn_out = self.output_layer(self.dnn(torch.cat([...], dim=1)))
        return linear_out + fm_out + dnn_out  # Sum all three terms
```

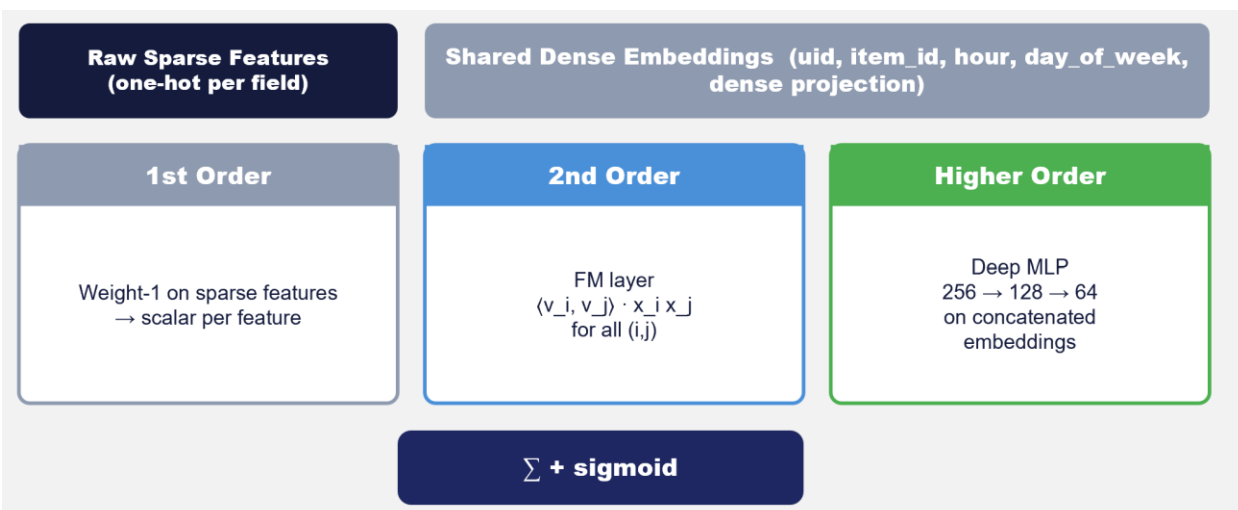

Figure 1: DeepFM architecture showing shared embeddings feeding into parallel FM (pairwise dot products) and Deep (MLP) components, with outputs summed through sigmoid

6.5.2 FM vs. Manual Feature Crosses

The FM component eliminates the manual feature engineering burden that Wide & Deep (Cheng et al., DLRS 2016) introduced. The original Wide & Deep paper (Cheng et al., "Wide & Deep Learning for Recommender Systems," Proceedings of the 1st Workshop on Deep Learning for Recommender Systems, 2016) describes how Google Play used the architecture to serve app recommendations to over one billion active users. In Wide & Deep, the 'wide' linear component requires hand-crafted cross-product features. DeepFM replaces this with the FM layer that automatically computes all pairwise interactions:

```
FM output = sum over all pairs (i,j) of <v_i, v_j> * x_i * x_j
```

where v_i is the embedding vector for feature i. The FM trick makes this computation efficient: O(n*k) instead of O(n^2*k) where n is the number of features and k is the embedding dimension.

6.5.3 When to Use DeepFM

- Well-suited for CTR prediction with many sparse categorical features (user IDs, item IDs, categories)
- Eliminates the need for manual feature cross engineering
- Relatively lightweight: the FM component adds minimal overhead compared to a standalone MLP
- A natural upgrade from XGBoost when you want end-to-end learning with embeddings

6.6 Ranking vs. Retrieval: Key Differences

Aspect	Retrieval (Chapter 5)	First-Stage Ranking (Chapter 6)
Candidate set	Millions to low billions of items	Hundreds to 1000s of items
Latency budget	~10ms	~50-100ms
Model complexity	Lightweight (two-tower, ANN)	Moderate (GBDT, DeepFM)
Feature types	User/item embeddings only	Cross-features, aggregates, context
Optimization	Recall@K	AUC, NDCG, precision
Key advantage	Speed and coverage	Precision and personalization

The fundamental tradeoff is between speed and precision. Retrieval must be fast enough to scan millions of items, so it uses simple scoring functions (dot products). The first-stage ranker has the luxury of operating on a reduced set, so it can compute richer feature combinations that would be prohibitive at retrieval scale.

6.7 Deployment Considerations for First-Stage Ranking

6.7.1 Latency and Throughput

First-stage ranking typically operates under a latency budget of 50-100 milliseconds. Key design decisions that affect serving latency:

- **Model complexity:** Tree depth, number of trees, and ensemble size directly impact inference time. During training, cap tree depth and number of estimators to balance accuracy vs. latency.
- **Feature computation:** Feature fetching often dominates end-to-end latency. Pre-computing and caching user features in a feature store (Chapter 4) is critical.
- **Batch scoring:** Score all candidates for a user in a single batch call rather than individually to amortize model loading and feature fetching overhead.

6.7.2 Model Serialization and Serving

- XGBoost models serialize to a compact binary format that loads in milliseconds
- Many serving frameworks (TensorFlow Serving, Triton, custom microservices) support XGBoost natively
- For DeepFM, export the trained model as ONNX or TorchScript for optimized inference
- Consider model distillation: train a complex model offline, then distill it into a simpler model for serving

6.7.3 When to Graduate to a Heavy Ranker

The first-stage ranker is not meant to be the final answer. It is a filter that ensures the most promising candidates proceed to a more sophisticated second stage. Consider adding a second-stage (heavy) ranker when:

- Your first-stage ranker's capacity is bottlenecked by latency constraints and you have measurable headroom in relevance
- You need multi-task predictions (e.g., P(click), P(like), P(purchase)) that a single-objective first-stage model cannot provide

- Cross-feature interactions between user history and candidate items require deeper modeling (attention mechanisms, sequence modeling)
- Your system has grown to the point where a single ranking pass cannot adequately serve both precision and recall needs

Chapter 7 covers the second-stage (heavy) ranker in detail, including DCN-V2, DLRM, and multi-task models like MMoE.

6.8 Code Tutorial: XGBoost Ranking on Yambda

The code tutorial for this chapter demonstrates first-stage ranking using the Yandex Yambda dataset, a large-scale music recommendation dataset with 50 million user-item interactions.

6.8.1 Dataset and Evaluation Protocol

Yambda dataset provides rich interaction signals: listen events with played ratio (how much of the track the user listened to), explicit feedback (likes, dislikes), and contextual information (organic vs. recommendation-driven discovery). Listen completion is defined as played_ratio >= 50%, a proxy for meaningful engagement.

We follow the Global Temporal Split (GTS) protocol: 300 days for training, a 30-minute gap to prevent information leakage, and 1 day for testing. This mirrors production practice where models are trained on historical data and evaluated on future interactions.

6.8.2 Feature Engineering Pipeline

The tutorial implements frequency encoding for high-cardinality features (user IDs, item IDs) and derives temporal features from the timestamp deltas. The feature encoder is designed to be serializable so the same transformations can be applied at inference time.

This matters because the single most common source of silent production bugs in ranking systems is train-serve skew: the model is trained on one definition of a feature but served on a slightly different one. A classic failure mode is recomputing frequency-encoding vocabularies at inference time from live data, which means a user whose ID was rare at training time gets mapped to a different bucket in production. By serializing the fitted encoder (vocabularies, bin edges, imputation values, scaler parameters) and loading the exact same object at inference, we guarantee that the function the model learned is the function it is being asked to apply. This property is also what makes a feature pipeline unit-testable: we can assert that transform(x) produces identical output across training and serving code paths.

6.8.3 Model Training and Evaluation

We train both a pointwise classifier (binary:logistic objective) and a pairwise ranker (rank:pairwise with LambdaMART) and compare their performance on AUC, log loss, and NDCG metrics. The tutorial also demonstrates hyperparameter tuning with early stopping and the impact of training data volume on model performance.

The training data volume sweep is worth running explicitly because it answers a question that shows up in every planning review: do we need more data, or a better model? Plotting the validation NDCG as the training set grows from 10%, 25%, 50%, to 100% of available interactions reveals whether the learning curve has plateaued or is still climbing. A flat curve means the model has saturated and further gains must come from better features, a more expressive architecture, or better labels; a steeply rising curve means the cheapest win is simply to log and train on more data. This diagnostic also helps scope infrastructure investments, because doubling the training set has very different ROI in the two regimes.

6.8.4 Key Takeaways from the Tutorial

- Correctly implementing Feature processing and doing feature engineering (especially cross-features and temporal aggregations) has more impact on ranking quality than specific algorithm choices
- XGBoost provides strong baselines that are hard to beat without significantly more complex models
- Feature importance analysis reveals which signals drive ranking decisions, guiding further feature investment

6.9 Chapter Summary

- The first-stage ranker bridges the gap between fast-but-coarse retrieval and precise-but-expensive second-stage ranking
- Learning to Rank provides three paradigms (pointwise, pairwise, listwise); pointwise is the pragmatic starting point for most systems
- Feature engineering is the highest-leverage activity: cross-features between user and item, temporal aggregations, and encoding strategies for high-cardinality features
- Gradient boosted trees (XGBoost, LightGBM) offer an excellent accuracy-to-complexity ratio for first-stage ranking
- DeepFM automates pairwise feature crosses while adding a deep component for higher-order interactions

- Deployment requires balancing model complexity against latency budgets, with feature fetching often being the true bottleneck
- When the first-stage ranker's capacity is exhausted, adding a second-stage heavy ranker (Chapter 7) with richer modeling is the natural next step.

References

- Guo, H., Tang, R., Ye, Y., Li, Z., & He, X. (2017). *DeepFM: A Factorization-Machine based Neural Network for CTR Prediction.* IJCAI 2017.
- Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System.* KDD 2016.
- Burges, C. J. C. (2010). *From RankNet to LambdaRank to LambdaMART: An Overview.* Microsoft Research Technical Report.
- Cheng, H.-T. et al. (2016). *Wide & Deep Learning for Recommender Systems.* DLRS Workshop, RecSys 2016.
- Meta Engineering. (2023). *Scaling Instagram Explore Recommendations.* engineering.fb.com.
- Yandex. (2025). *Yambda-5B: A Large-Scale Multi-modal Dataset for Ranking and Retrieval.* arXiv:2505.22238.
- Ke, G. et al. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree.* NeurIPS 2017.